Software Configuration Standard - (SCFS)

Software Configuration Standard - (SCFS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Configuration Governance

Standard ID: SLGA-SCFS-004

Origin ID: OOF.PS.SLGA.04.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 12.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Software Configuration Standard (SCFS) governs the identification,

establishment, authorization, composition, preservation, validation, change, and lifecycle continuity of software configuration states.

SCFS establishes the methodological conditions under which software

configuration can be recognized as an identifiable, bounded, evidence-supported, and governable representation of how software components, artifacts, versions, parameters, settings, and operational relationships are assembled and intended to function within a defined software context.

It governs the relationship between software configuration objects,

configuration baselines, version relationships, artifact configurations, configuration parameters, environmental assumptions, authorized configuration changes, actual configuration states, and the evidence required to establish configuration integrity and continuity.

SCFS distinguishes configuration documentation from actual configuration,

component availability from configuration validity, configuration consistency from operational correctness, and authorized configuration intent from verified configuration realization.

It applies to software developed, configured, maintained, or operated by

humans, organizations, automated systems, AI coding assistants, autonomous development agents, and software infrastructure mechanisms.

Its principal output is:

Governed Software Configuration State

Its foundational distinction is:

Declared Software Configuration ≠ Governed Software Configuration State.

Operational Role

The operational role of SCFS is:

To ensure that software configuration remains identifiable, authorized, internally coherent, evidence-supported, traceable to its constituent components and settings, and verifiably aligned with the configuration state actually established throughout the software lifecycle.

SCFS establishes the configuration-governance foundation of SLGA™.

It governs how software configuration states are defined, assembled, controlled, validated, and preserved.

It does not independently establish software requirements, authorize dependency introduction, certify software quality or security, approve release, or authorize deployment.

Its outputs provide the configuration evidence and state relationships

required by competent downstream lifecycle governance.

Core Question

How can software configuration remain identifiable, authorized, consistent, verifiable, and traceable across components, versions, artifacts, environments, and lifecycle changes—without confusing declared configuration with the software configuration actually established?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

SCFS governs:

Software Configuration

It is the fourth Parent Standard of SLGA™.

Its principal upstream references include:

Software Requirements Standard (SRS)

Software Creation Standard (SCS)

Software Dependency Standard (SDS)

Its principal downstream relationships include:

Software Quality Standard (SQS)

Software Security Standard (SSS)

Software Release Standard (SRLS)

Software Deployment Standard (SDPS)

Software Change Management Standard (SCMS)

Software Retirement Standard (SRTS)

Software Lifecycle Control Standard (SLCS)

SCFS provides the configuration state against which relevant quality,

security, release, deployment, and lifecycle-control assessments may be performed.

Configuration Governance ≠ Release Authorization.

• Governed Space

Software Configuration Governance

This space governs the material conditions under which software

configuration states are established, represented, controlled, and validated.

Its principal dimensions include:

• Configuration object identity

• Configuration scope

• Configuration purpose

• Configuration items

• Configuration relationships

• Configuration baselines

• Configuration baseline authority

• Configuration versions

• Version relationships

• Artifact configuration

• Configuration parameters

• Configuration values

• Environment-specific configuration

• Configuration constraints

• Configuration assumptions

• Configuration composition

• Configuration consistency

• Configuration completeness

• Configuration change

• Configuration change authority

• Configuration change effects

• Declared configuration

• Actual configuration

• Configuration evidence

• Configuration validation

• Configuration drift

• Configuration conflict

• Configuration gaps

• Configuration reconstruction

• Configuration rollback references

• Configuration continuity

• Configuration governance states

The governed space is technology-independent and applies across software development, testing, release preparation, deployment preparation, operation, maintenance, and retirement.

Software Configuration Governance Object

Every material SCFS assessment shall establish an identifiable Software

Configuration Governance Object.

The object should identify:

Configuration Object Identity: The configuration state or configuration

relationship under assessment.

Software Object: The software system, application, service, subsystem, or

artifact concerned.

Configuration Purpose: The function or operational condition the

configuration supports.

Configuration Scope: The components, parameters, environments, and

relationships included.

Configuration Items: The material software and configuration elements.

Configuration Baseline Reference: The applicable approved or proposed

baseline.

Version Relationships: Relevant component and artifact versions.

Configuration Parameters: Material settings and values.

Environment Reference: The environment in which the configuration is intended or observed.

Authority Reference: The applicable basis for configuration establishment

or change.

Declared Configuration: The specified or expected configuration state.

Actual Configuration: The configuration state supported by observation or

evidence.

Configuration Evidence: Records supporting material configuration claims.

Configuration Uncertainty: Material unresolved configuration conditions.

Governance State: The recognized configuration condition.

A configuration governance object shall remain distinguishable from a

configuration file, repository record, or deployed software instance.

Configuration File ≠ Software Configuration Governance Object.

Configuration Governance Principles

SCFS establishes the following principles. 7.1 Identifiability

Material configuration states shall be sufficiently identifiable. 7.2 Contextuality

Configuration validity shall be assessed within its intended software and environment context. 7.3 Authority

Material configuration establishment and change shall remain subject to

applicable authority. 7.4 Composition Integrity

Configuration relationships shall remain sufficiently coherent and

reconstructable. 7.5 Evidence

Material configuration claims shall be supported by competent evidence. 7.6 Declared–Actual Distinction

Declared configuration shall remain distinguishable from the configuration

actually established. 7.7 Change Traceability

Material configuration changes shall remain traceable to their relevant

basis and resulting state. 7.8 Continuity

Configuration identity, history, and relevant evidence shall be preserved

across material lifecycle transitions. 7.9 Uncertainty Preservation

Unresolved configuration conditions shall not be represented as verified

facts. 7.10 Lifecycle Interoperability

Configuration outputs shall support competent downstream SLGA™ governance

without replacing it.

Configuration Object Identification

SCFS shall identify the software object and configuration context being

governed.

Relevant software objects may include:

• Applications

• Services

• Libraries

• Executable artifacts

• Containers

• Software subsystems

• Distributed software components

• Build outputs

• Runtime instances

• Configuration packages

The same software artifact may operate under different configurations.

Software Artifact Identity ≠ Software Configuration Identity.

Configuration Scope

SCFS shall establish the boundaries of the configuration state under

assessment.

Scope may include:

• Software components

• Artifact versions

• Runtime settings

• Feature settings

• Interface parameters

• Environment variables

• Service endpoints

• Configuration files

• Deployment-related settings

• Dependency resolution references

Scope shall distinguish included, excluded, and unresolved material

configuration elements.

Configuration Scope Definition ≠ Configuration Completeness.

Configuration Item Recognition

SCFS shall recognize material configuration items and their roles.

A configuration item may be:

• A software artifact

• A component version

• A configuration file

• A parameter set

• A runtime setting

• A service integration reference

• A build configuration

• A platform-specific setting

• A feature-control setting

Configuration items shall remain identifiable in relation to the software

configuration state.

Configuration Item Presence ≠ Configuration Item Validity.

Configuration Relationship Governance

SCFS shall establish material relationships between configuration items.

Relationships may include:

• Component-to-version relationships

• Artifact-to-parameter relationships

• Service-to-endpoint relationships

• Runtime-to-environment relationships

• Feature-to-configuration relationships

• Build-to-artifact relationships

• Baseline-to-configuration relationships

A set of valid configuration items may still form an inconsistent configuration.

Individually Valid Configuration Items ≠ Valid Configuration Composition.

Configuration Purpose

SCFS shall identify the purpose of material configuration conditions.

Purpose may relate to:

• Enabling software functionality

• Selecting operational behavior

• Establishing environment compatibility

• Supporting integration

• Defining feature availability

• Supporting performance requirements

• Establishing software execution conditions

Configuration purpose shall remain distinguishable from configuration

effect.

Intended Configuration Purpose ≠ Actual Configuration Effect.

Configuration Baseline Governance

SCFS shall establish identifiable configuration baselines where material to software governance.

A configuration baseline represents a defined configuration reference

against which subsequent states or changes may be assessed.

A baseline should identify:

• Relevant software object

• Configuration scope

• Constituent configuration items

• Version relationships

• Applicable parameters

• Intended environment

• Baseline authority

• Baseline evidence

• Temporal applicability

A baseline shall not automatically be treated as technically correct merely

because it was approved.

Approved Configuration Baseline ≠ Verified Configuration Correctness.

Baseline Establishment Authority

SCFS shall identify the legitimate basis for establishing or modifying a

configuration baseline.

Relevant conditions may include:

• Competent organizational authority

• Approved configuration responsibility

• Delegated software maintenance authority

• Defined change-management conditions

• Applicable software lifecycle decisions

Authority shall be bounded by software scope, action type, and applicable

conditions.

Authority to Configure Software ≠ Unlimited Authority to Change Every

Configuration.

Configuration Baseline Integrity

SCFS shall preserve the integrity of material configuration baseline

records.

Relevant considerations may include:

• Baseline identity

• Content integrity

• Constituent item references

• Version correspondence

• Authorization evidence

• Change history

• Temporal relevance

An intact baseline record does not establish that the actual software environment conforms to it.

Baseline Record Integrity ≠ Actual Configuration Conformity.

Configuration Version Governance

SCFS shall govern material version relationships within software configuration.

Version relationships may involve:

• Application versions

• Component versions

• Runtime versions

• Configuration schema versions

• Build versions

• Interface versions

• Environment-specific configuration revisions

Version identity shall remain distinguishable from version compatibility.

Matching Version Labels ≠ Demonstrated Configuration Compatibility.

Version Relationship Consistency

SCFS shall assess material consistency among versioned configuration items.

Relevant concerns may include:

• Incompatible component versions

• Incorrect configuration schema versions

• Unsupported runtime relationships

• Conflicting version references

• Unresolved artifact-version correspondence

Consistency shall be assessed against the actual configuration context.

Version Record Consistency ≠ Complete Software Configuration Validity.

Software Artifact Configuration

SCFS shall govern material configuration relationships associated with

software artifacts.

Relevant artifacts may include:

• Executables

• Libraries

• Containers

• Packages

• Generated outputs

• Deployable bundles

• Runtime images

Artifact configuration may concern:

• Included components

• Referenced versions

• Build parameters

• Runtime assumptions

• Configuration metadata

• Relevant external resources

Artifact identity shall not be treated as proof of complete configuration

identity.

Identical Software Artifact ≠ Identical Operational Configuration.

Configuration Parameter Governance

SCFS shall identify material software configuration parameters.

Parameters may influence:

• Software behavior

• Feature activation

• Resource use

• Service integration

• Execution modes

• Logging behavior

• Interface behavior

• Runtime operation

Parameter governance shall preserve relevant identity, value, scope, and

context.

Parameter Definition ≠ Verified Parameter Application.

Configuration Value Governance

SCFS shall assess whether material configuration values are consistent with the intended configuration context.

Relevant considerations may include:

• Value identity

• Applicable range

• Required format

• Environmental constraints

• Inter-parameter relationships

• Version correspondence

• Material defaults

• Effective overrides

A syntactically valid configuration value may still be operationally inappropriate.

Syntactically Valid Configuration ≠ Contextually Valid Configuration.

Default Configuration Recognition

SCFS shall recognize material default configuration values and behaviors.

Defaults may originate from:

• Software frameworks

• Libraries

• Runtime platforms

• Configuration schemas

• Build tools

• Infrastructure services

Defaults shall not be presumed appropriate merely because they are supplied by the software producer.

Default Configuration ≠ Approved Configuration.

Configuration Override Governance

SCFS shall identify material configuration overrides.

Overrides may arise through:

• Environment variables

• Command-line arguments

• Runtime configuration services

• Deployment settings

• User-specific settings

• Automated configuration mechanisms

The precedence and effect of overrides shall remain reconstructable where material.

Declared Base Configuration ≠ Effective Configuration After Overrides.

Environment-Specific Configuration

SCFS shall distinguish configuration states across software environments.

Relevant environments may include:

• Development

• Testing

• Staging

• Production

• Maintenance

• Recovery

A configuration valid in one environment may be invalid in another.

Development Configuration Validity ≠ Production Configuration Validity.

Configuration Environment Assumptions

SCFS shall identify material assumptions about the environment in which

configuration is intended to operate.

Assumptions may concern:

• Runtime capabilities

• Platform versions

• Available services

• Resource limits

• Network conditions

• External endpoints

• Required configuration mechanisms

Unsupported assumptions shall remain explicit.

Assumed Environment Compatibility ≠ Verified Configuration Compatibility.

Configuration Composition

SCFS shall govern how configuration items combine into a material software

configuration state.

Composition may involve:

• Component versions

• Runtime parameters

• Feature settings

• Interface configuration

• External service references

• Build settings

• Environment-specific values

Composition shall preserve the material relationships necessary to

understand the resulting configuration.

Configuration Item Collection ≠ Governed Configuration Composition.

Configuration Consistency

SCFS shall assess material consistency within and across configuration items.

Consistency may concern:

• Parameter relationships

• Version relationships

• Artifact references

• Environment settings

• Interface configuration

• Dependency references

• Baseline correspondence

Consistency shall not be inferred solely from successful parsing or startup.

Configuration Parse Success ≠ Configuration Consistency.

Configuration Completeness

SCFS shall assess whether the material configuration scope has been

sufficiently represented.

Completeness may concern:

• Required configuration items

• Required parameter values

• Relevant defaults

• Effective overrides

• Material external references

• Environment-specific settings

• Version relationships

Completeness shall be assessed relative to a defined purpose and scope.

Complete Configuration Documentation ≠ Verified Actual Configuration.

Configuration Constraints

SCFS shall identify material constraints governing software configuration.

Constraints may arise from:

• Software requirements

• Dependency compatibility

• Platform conditions

• Operational requirements

• Security requirements

• Performance requirements

• Release conditions

• Deployment conditions

SCFS preserves configuration-related constraints without replacing

competent quality, security, or deployment assessment.

Configuration Constraint Satisfaction ≠ Complete Software Compliance.

Declared Configuration

SCFS shall establish the configuration state represented by authoritative

or proposed configuration declarations.

Declared configuration may be represented through:

• Configuration files

• Build manifests

• Deployment manifests

• Configuration databases

• Infrastructure definitions

• Parameter records

• Baseline specifications

A declaration is evidence of intended or recorded configuration, not necessarily of actual configuration.

Declared Configuration ≠ Actual Configuration.

Actual Configuration

SCFS shall recognize the configuration state supported by competent

observation or evidence.

Actual configuration may differ from declared configuration because of:

• Runtime overrides

• Manual changes

• Automated changes

• Environment-specific defaults

• Incomplete deployment

• Configuration propagation delays

• External service changes

• Unrecorded modifications

Actual configuration shall be distinguished from assumptions about the

intended state.

Expected Configuration ≠ Observed Configuration.

Declared–Actual Configuration Alignment

SCFS shall assess material alignment between declared and actual

configuration states.

Relevant comparisons may include:

• Component versions

• Artifact references

• Effective parameters

• Runtime settings

• Service endpoints

• Feature states

• Environment conditions

Alignment shall be determined within the assessed scope and evidence limitations.

Declared–Actual Alignment ≠ Guaranteed Software Correctness.

Configuration Change Governance

SCFS shall govern material changes to software configuration states.

Changes may include:

• Parameter modification

• Version reference changes

• Baseline changes

• Configuration item addition

• Configuration item removal

• Environment setting changes

• Artifact configuration changes

• Configuration override changes

Configuration changes shall remain traceable to their applicable authority

and resulting state.

Configuration Change Capability ≠ Configuration Change Authority.

Configuration Change Authority

SCFS shall identify the legitimate basis for material configuration changes.

Authority shall be evaluated relative to:

• Actor or execution mechanism

• Software object

• Configuration item

• Change type

• Environment

• Scope

• Applicable restrictions

• Temporal validity

An automated system may be capable of changing configuration without possessing unlimited authority to do so.

Automated Configuration Capability ≠ Autonomous Configuration Authority.

Configuration Change Proposal

SCFS shall distinguish proposed configuration changes from authorized and

executed changes.

A proposal should identify:

• Proposed modification

• Configuration object

• Intended purpose

• Affected items

• Applicable baseline

• Expected effects

• Required authority

• Relevant constraints

A proposal shall not automatically become an approved configuration state.

Proposed Configuration ≠ Authorized Configuration.

Configuration Change Execution

SCFS shall preserve the relationship between an authorized configuration

change and its actual execution.

Execution evidence may include:

• Configuration file changes

• Repository records

• Deployment configuration events

• Runtime configuration observations

• Automation logs

• Configuration service records

The execution record shall remain distinguishable from the resulting

effective configuration.

Executed Configuration Change ≠ Verified Effective Configuration Change.

Configuration Change Effects

SCFS shall identify material direct and indirect effects of configuration

changes.

Effects may include:

• Changed software behavior

• Altered service connectivity

• Changed feature availability

• Modified runtime resource use

• Changed component compatibility

• Altered operational assumptions

• New configuration conflicts

A configuration change may affect multiple software components.

Localized Configuration Modification ≠ Localized Configuration Effect.

Configuration Change Traceability

SCFS shall preserve material relationships between configuration changes

and their:

• Purpose

• Authority

• Affected items

• Previous state

• Executed action

• Resulting state

• Supporting evidence

• Subsequent changes

Traceability shall support reconstruction of material configuration

transitions.

Configuration Change Log ≠ Complete Configuration Change Traceability.

Configuration Change Continuity

SCFS shall preserve continuity across material configuration changes.

Continuity may require:

• Historical baseline references

• Previous parameter values

• Version history

• Change authorization records

• Resulting-state evidence

• Supersession relationships

• Relevant rollback references

Current configuration shall not erase material historical configuration

relationships.

Current Configuration State ≠ Complete Configuration History.

Configuration Validation

SCFS shall establish whether material configuration conditions are sufficiently supported.

Validation may consider:

• Configuration identity

• Baseline correspondence

• Version consistency

• Parameter validity

• Composition integrity

• Environment alignment

• Declared–actual correspondence

• Change authority

• Evidence competence

• Material uncertainty

Validation shall remain limited to the configuration conditions actually

assessed.

Validated Configuration State ≠ Validated Software Product.

Configuration Evidence

SCFS shall preserve evidence supporting material configuration claims.

Evidence may include:

• Configuration manifests

• Baseline records

• Artifact metadata

• Build records

• Version records

• Runtime observations

• Configuration snapshots

• Automation logs

• Change records

• Environment assessments

Evidence shall remain associated with the configuration claim it supports.

Configuration Evidence Presence ≠ Configuration Claim Validity.

Configuration Evidence Competence

SCFS shall assess whether evidence is competent to support the relevant configuration determination.

For example:

A configuration file may support a declared setting.

A runtime observation may support an effective setting.

A build manifest may support an intended artifact composition.

An artifact inspection may support the observed artifact composition.

These records establish different facts.

Evidence of Declared State ≠ Evidence of Effective State.

Configuration Evidence Integrity

SCFS shall preserve the integrity of material configuration evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Temporal relevance

• Artifact correspondence

• Environment correspondence

• Configuration scope

• Preservation history

• Material contradictions

An intact configuration record does not independently establish that the configuration is suitable for its intended use.

Intact Configuration Evidence ≠ Configuration Suitability Assurance.

Configuration Drift

Configuration Drift occurs when the actual or effective configuration materially diverges from an applicable reference configuration or previously established state.

Drift may arise through:

• Manual modifications

• Automated changes

• Runtime overrides

• Platform updates

• Configuration propagation

• Unrecorded environment changes

• Supplier-controlled settings

Drift shall be assessed against an identifiable reference.

Configuration Difference ≠ Automatically Unauthorized Configuration Drift.

Configuration Conflict

A Configuration Conflict exists where material configuration requirements, values, references, or evidence are incompatible.

Examples may include:

• Contradictory parameter values

• Incompatible version references

• Conflicting environment settings

• Competing configuration sources

• Inconsistent artifact configuration claims

SCFS shall preserve the conflict and its supporting evidence.

Conflicting Configuration Records ≠ Permission to Select the Preferred

Configuration.

Configuration Gap

A Configuration Gap exists where material configuration information is missing or insufficiently established.

Examples may include:

• Unknown effective parameter values

• Missing baseline references

• Unresolved version relationships

• Incomplete configuration composition

• Unverified runtime overrides

• Missing configuration change evidence

SCFS shall identify the gap and its material significance.

Configuration Evidence Gap ≠ Proven Configuration Failure.

Critical Configuration Gap

A Critical Configuration Gap exists where missing configuration information prevents competent assessment of a materially consequential software configuration.

Examples may include:

• Unknown effective configuration of a critical production service

• Unverified artifact versions in a release candidate

• Unresolved runtime overrides affecting essential functionality

• Missing configuration evidence for a safety-relevant software

component.

SCFS shall preserve the critical gap for competent assessment.

It does not independently determine operational shutdown, security compromise, or legal liability.

Critical Configuration Gap ≠ Automatic Software Failure.

Configuration Uncertainty

SCFS shall preserve material uncertainty concerning configuration identity,

composition, validity, authority, or effective state.

Uncertainty may arise where:

• Actual configuration cannot be observed

• Baseline applicability is unclear

• Configuration precedence is unresolved

• Artifact correspondence is uncertain

• Environment assumptions are unsupported

• Change authority is ambiguous

Uncertainty shall not be silently converted into configuration acceptance.

Unresolved Configuration ≠ Verified Configuration.

Configuration Reconstruction

SCFS shall support reconstruction of material configuration states where sufficient evidence exists.

Reconstruction may use:

• Baseline records

• Version references

• Configuration snapshots

• Artifact metadata

• Change histories

• Runtime observations

• Relevant environmental records

Reconstructed states shall distinguish demonstrated facts from assumptions

and unresolved relationships.

Reconstructed Configuration ≠ Directly Observed Configuration.

Configuration Rollback Reference

SCFS shall preserve relevant configuration rollback references where applicable.

Rollback information may include:

• Previous baseline

• Previous parameter values

• Previous artifact versions

• Required environmental conditions

• Relevant change records

• Verification requirements

Rollback execution and restoration of actual software operation remain

subject to competent lifecycle governance.

Configuration Rollback Command ≠ Verified Configuration Restoration.

Configuration Continuity

SCFS shall preserve material configuration identity and state relationships

throughout the software lifecycle.

Continuity may involve:

• Requirements-to-configuration relationships

• Creation-to-configuration relationships

• Dependency-to-configuration relationships

• Quality assessment references

• Security assessment references

• Release configuration references

• Deployment configuration references

• Change history

• Retirement references

Configuration continuity shall preserve the distinction between historical,

current, proposed, and actual states.

Configuration Continuity ≠ Configuration Immutability.

Configuration Governance States

SCFS shall support identifiable configuration governance states.

Possible states include:

Identified: A configuration object has been recognized.

Proposed: A configuration state has been specified but not established.

Under Assessment: Material configuration conditions are being evaluated.

Baseline-Established: An identifiable configuration baseline has been

established.

Conditionally Validated: Configuration validation depends on explicit

conditions.

Validated: Material configuration conditions are sufficiently supported within the assessed scope.

Partially Validated: Some material configuration conditions remain

unresolved.

Evidence-Limited: Available evidence is insufficient for the required determination.

Authority-Unresolved: Applicable configuration authority remains

uncertain.

Version-Unresolved: Material version relationships remain unresolved.

Composition-Unresolved: Material configuration composition remains

insufficiently established.

Environment-Unresolved: Relevant environmental configuration conditions

remain uncertain.

Drift-Detected: Material divergence from an applicable reference has been

identified.

Conflicted: Material configuration claims or evidence are contradictory.

Invalid: The configuration does not satisfy material requirements within the assessed scope.

Superseded: A later configuration state has replaced the earlier state.

Retired: The configuration is no longer an active applicable state.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not independently establish software quality, security,

release readiness, or deployment authorization.

Configuration Validated ≠ Software Release Approved.

Core Modules

SCFS is implemented through five Core Modules. 52.1 CBM — Configuration Baseline Module

Governed Mechanism: Configuration baseline establishment, identity,

authority, preservation, and reference integrity.

CBM governs how software configuration baselines are defined, established, authorized, and preserved as identifiable reference states.

It distinguishes approved baseline records from verified actual

configuration.

Primary Output:

Governed Configuration Baseline State

52.2 BMM — Build Management Module

Governed Mechanism: Software build configuration, build inputs, execution

conditions, and build-to-artifact correspondence.

BMM governs the configuration relationships required to establish how

software build outputs correspond to identified build inputs, parameters, tools, and conditions.

It distinguishes build execution success from governed build configuration

and artifact correspondence.

Primary Output:

Governed Software Build State

52.3 AVIM — Artifact Version Integrity Module

Governed Mechanism: Software artifact version identity, version

relationships, correspondence, and integrity.

AVIM governs the relationship between identified software artifacts, their

versions, relevant configuration states, and the evidence supporting artifact-version integrity.

It distinguishes artifact version labels from verified artifact-version

identity and correspondence.

Primary Output:

Governed Artifact Version Integrity State

52.4 CCCM — Configuration Change Control Module

Governed Mechanism: Configuration change authority, scope, execution,

resulting-state correspondence, and change continuity.

CCCM governs the legitimate modification of software configuration states

and the relationship between proposed, authorized, executed, and observed configuration changes.

It distinguishes configuration modification capability from legitimate

configuration change authority.

Primary Output:

Governed Configuration Change State

52.5 BREM — Build Reproducibility & Environment Module

Governed Mechanism: Build environment identity, environmental

assumptions, reproducibility conditions, and evidence-supported build reconstruction.

BREM governs whether software build results can be meaningfully reproduced or reconstructed under sufficiently identified and comparable build conditions.

It distinguishes repeated build success from demonstrated build

reproducibility.

Primary Output:

Governed Build Reproducibility & Environment State

Core Module Governance Flow

The SCFS internal governance flow is:

Configuration Baseline → Build Management → Artifact Version Integrity →

Configuration Change Control → Build Reproducibility & Environment

The corresponding module sequence is:

CBM → BMM → AVIM → CCCM → BREM

This sequence represents the principal methodological relationship among

the five modules.

It is not a mandatory one-directional implementation workflow.

Configuration changes may require baseline revision.

Artifact-version integrity findings may require build reassessment.

Reproducibility findings may reveal unresolved build configuration or environmental assumptions.

The modules shall support iterative reassessment while preserving their

distinct governed mechanisms.

Configuration Governance Requirements

SCFS establishes the following requirements.

SCFS-GR-01 — Configuration Object Identification

Material software configuration states shall be sufficiently identifiable.

SCFS-GR-02 — Configuration Scope

The material scope of each configuration assessment shall be defined.

SCFS-GR-03 — Configuration Item Recognition

Material configuration items and their roles shall be recognized.

SCFS-GR-04 — Configuration Relationship Integrity

Material relationships between configuration items shall remain

sufficiently established.

SCFS-GR-05 — Configuration Purpose

Material configuration conditions shall remain associated with their

intended software purpose.

SCFS-GR-06 — Configuration Baseline

Applicable configuration baselines shall be identifiable and

evidence-supported.

SCFS-GR-07 — Configuration Authority

Material configuration establishment and change shall remain subject to

applicable authority.

SCFS-GR-08 — Configuration Version Integrity

Material version relationships shall remain identifiable and sufficiently

supported.

SCFS-GR-09 — Build Configuration

Material build inputs, parameters, tools, and conditions shall remain

traceable to relevant build outputs.

SCFS-GR-10 — Artifact Version Correspondence

Software artifacts shall remain traceable to their relevant versions and

configuration conditions.

SCFS-GR-11 — Configuration Composition

Material configuration composition shall remain sufficiently coherent and

reconstructable.

SCFS-GR-12 — Configuration Parameters

Material parameters, values, defaults, and overrides shall remain

identifiable where relevant.

SCFS-GR-13 — Environment Context

Configuration validity shall be assessed within the applicable software environment.

SCFS-GR-14 — Declared–Actual Alignment

Declared configuration shall remain distinguishable from actual

configuration, and material differences shall be recognized.

SCFS-GR-15 — Configuration Change Control

Material configuration changes shall remain attributable to applicable

authority, scope, execution, and resulting state.

SCFS-GR-16 — Configuration Validation

Material configuration claims shall be subject to evidence-supported

validation.

SCFS-GR-17 — Configuration Drift

Material divergence from applicable configuration references shall be

recognized and assessed.

SCFS-GR-18 — Configuration Evidence

Material configuration evidence shall remain associated with the claims it

supports.

SCFS-GR-19 — Configuration Uncertainty and Conflict

Material uncertainty, gaps, and conflicting configuration evidence shall

remain explicit.

SCFS-GR-20 — Build Reproducibility

Material build reproducibility claims shall be supported by sufficiently

identified build and environment conditions.

SCFS-GR-21 — Configuration Continuity

Material configuration identity, history, and evidence shall remain

reconstructable across relevant lifecycle changes.

SCFS-GR-22 — Lifecycle Handoff

Configuration governance outputs shall remain sufficiently structured for

competent downstream SLGA™ governance.

Configuration Governance Logic

The canonical SCFS governance logic is:

Receive Governed Software Requirements Context

↓

Receive Governed Software Creation Context

↓

Receive Governed Software Dependency State

↓

Identify Software Configuration Governance Object

↓

Establish Configuration Purpose and Scope

↓

Identify Material Configuration Items

↓

Establish Configuration Relationships

↓

Identify Applicable Configuration Authority

↓

Establish Configuration Baseline

↓

Identify Software Versions and Configuration Parameters

↓

Identify Build Inputs, Tools, and Conditions

↓

Establish Build-to-Artifact Correspondence

↓

Assess Artifact Version Identity and Integrity

↓

Establish Configuration Composition

↓

Identify Environment-Specific Configuration

↓

Identify Defaults, Overrides, and Constraints

↓

Establish Declared Configuration State

↓

Establish Evidence-Supported Actual Configuration State

↓

Assess Declared–Actual Alignment

↓

Identify Material Configuration Changes

↓

Assess Configuration Change Authority and Execution

↓

Assess Configuration Consistency and Completeness

↓

Assess Build Reproducibility and Environment Conditions

↓

Associate Configuration Evidence

↓

Validate Material Configuration Conditions

↓

Identify Configuration Drift

↓

Identify Configuration Gaps, Conflicts, and Uncertainty

↓

Determine Software Configuration Governance State

↓

Establish Governed Software Configuration State

↓

Preserve Configuration Continuity

↓

Handoff to Competent SLGA™ Governance

↓

Reassess When Material Configuration Conditions Change

The principal output is:

Governed Software Configuration State

Governed Software Configuration State

The principal output of SCFS is:

Governed Software Configuration State

The state shall preserve sufficient information to identify:

Configuration Governance Object: The configuration assessment.

Software Object: The software system or artifact concerned.

Configuration Purpose: The intended configuration function.

Configuration Scope: The boundaries of the assessed configuration.

Configuration Items: The material constituent elements.

Configuration Relationships: The relationships among configuration items.

Configuration Baseline: The applicable reference configuration.

Baseline Authority: The authority basis for baseline establishment.

Version Relationships: Relevant software, component, and artifact

versions.

Build Configuration: Relevant build inputs, tools, parameters, and

conditions.

Build Output Correspondence: The relationship between build conditions

and resulting artifacts.

Artifact Version Integrity: Relevant artifact identity and version

evidence.

Configuration Parameters: Material configuration settings.

Configuration Values: Relevant parameter values.

Default Configuration: Applicable default settings.

Configuration Overrides: Material overrides and their effects.

Environment Context: The intended or observed environment.

Configuration Constraints: Applicable limits and conditions.

Configuration Composition: The resulting configuration relationships.

Configuration Consistency: The assessed internal coherence.

Configuration Completeness: The assessed completeness within scope.

Declared Configuration: The specified or expected state.

Actual Configuration: The evidence-supported effective state.

Declared–Actual Alignment: The correspondence between intended and actual

states.

Configuration Changes: Material modifications and their history.

Change Authority: The applicable authorization conditions.

Change Execution: The actions actually performed.

Change Effects: Material resulting configuration consequences.

Configuration Evidence: Evidence supporting material claims.

Evidence Competence: The suitability of evidence for the assessed claims.

Evidence Integrity: The condition of supporting evidence.

Configuration Validation: The scope and result of configuration

assessment.

Configuration Drift: Material divergence from applicable references.

Configuration Conflicts: Contradictory configuration conditions or

evidence.

Configuration Gaps: Missing material configuration information.

Critical Configuration Gaps: Material gaps requiring competent attention.

Configuration Uncertainty: Material unresolved conditions.

Configuration Reconstruction: Relevant historical reconstruction

information.

Rollback References: Relevant restoration references.

Build Reproducibility: The assessed reproducibility conditions.

Configuration Continuity: Preservation of material configuration

relationships across lifecycle changes.

Configuration Governance State: The recognized configuration condition.

Downstream References: Information required by competent SLGA™ governance.

The state shall not independently establish software product quality,

security certification, release authorization, or deployment authorization.

Governance Outputs

SCFS may produce:

Software Configuration Governance Object

Configuration Object Identification Record

Configuration Scope Record

Configuration Item Register

Configuration Relationship Map

Configuration Purpose Record

Configuration Baseline Record

Baseline Authority Record

Configuration Baseline Integrity Record

Configuration Version Register

Version Relationship Consistency Assessment

Software Artifact Configuration Record

Configuration Parameter Register

Configuration Value Assessment

Default Configuration Record

Configuration Override Record

Environment-Specific Configuration Record

Configuration Environment Assumption Record

Configuration Composition Record

Configuration Consistency Assessment

Configuration Completeness Assessment

Configuration Constraint Record

Declared Configuration State

Actual Configuration State

Declared–Actual Configuration Alignment Record

Configuration Change Record

Configuration Change Authority Record

Configuration Change Proposal

Configuration Change Execution Record

Configuration Change Effects Record

Configuration Change Traceability Record

Configuration Change Continuity Record

Configuration Validation Record

Configuration Evidence Register

Configuration Evidence Competence Assessment

Configuration Evidence Integrity Assessment

Configuration Drift Record

Configuration Conflict Record

Configuration Gap

Critical Configuration Gap

Configuration Uncertainty State

Configuration Reconstruction Record

Configuration Rollback Reference

Configuration Continuity Record

Configuration Governance State

Governed Configuration Baseline State

Governed Software Build State

Governed Artifact Version Integrity State

Governed Configuration Change State

Governed Build Reproducibility & Environment State

SLGA Configuration Handoff Record

The principal output is:

Governed Software Configuration State

Critical Distinctions

SCFS preserves the following distinctions:

Declared Software Configuration ≠ Governed Software Configuration State

Configuration Governance ≠ Release Authorization

Configuration File ≠ Software Configuration Governance Object

Software Artifact Identity ≠ Software Configuration Identity

Configuration Scope Definition ≠ Configuration Completeness

Configuration Item Presence ≠ Configuration Item Validity

Individually Valid Configuration Items ≠ Valid Configuration Composition

Intended Configuration Purpose ≠ Actual Configuration Effect

Approved Configuration Baseline ≠ Verified Configuration Correctness

Authority to Configure Software ≠ Unlimited Authority to Change Every

Configuration

Baseline Record Integrity ≠ Actual Configuration Conformity

Matching Version Labels ≠ Demonstrated Configuration Compatibility

Version Record Consistency ≠ Complete Software Configuration Validity

Identical Software Artifact ≠ Identical Operational Configuration

Parameter Definition ≠ Verified Parameter Application

Syntactically Valid Configuration ≠ Contextually Valid Configuration

Default Configuration ≠ Approved Configuration

Declared Base Configuration ≠ Effective Configuration After Overrides

Development Configuration Validity ≠ Production Configuration Validity

Assumed Environment Compatibility ≠ Verified Configuration Compatibility

Configuration Item Collection ≠ Governed Configuration Composition

Configuration Parse Success ≠ Configuration Consistency

Complete Configuration Documentation ≠ Verified Actual Configuration

Configuration Constraint Satisfaction ≠ Complete Software Compliance

Declared Configuration ≠ Actual Configuration

Expected Configuration ≠ Observed Configuration

Declared–Actual Alignment ≠ Guaranteed Software Correctness

Configuration Change Capability ≠ Configuration Change Authority

Automated Configuration Capability ≠ Autonomous Configuration Authority

Proposed Configuration ≠ Authorized Configuration

Executed Configuration Change ≠ Verified Effective Configuration Change

Localized Configuration Modification ≠ Localized Configuration Effect

Configuration Change Log ≠ Complete Configuration Change Traceability

Current Configuration State ≠ Complete Configuration History

Validated Configuration State ≠ Validated Software Product

Configuration Evidence Presence ≠ Configuration Claim Validity

Evidence of Declared State ≠ Evidence of Effective State

Intact Configuration Evidence ≠ Configuration Suitability Assurance

Configuration Difference ≠ Automatically Unauthorized Configuration Drift

Conflicting Configuration Records ≠ Permission to Select the Preferred

Configuration

Configuration Evidence Gap ≠ Proven Configuration Failure

Critical Configuration Gap ≠ Automatic Software Failure

Unresolved Configuration ≠ Verified Configuration

Reconstructed Configuration ≠ Directly Observed Configuration

Configuration Rollback Command ≠ Verified Configuration Restoration

Configuration Continuity ≠ Configuration Immutability

Configuration Validated ≠ Software Release Approved

Cross-Architecture Boundary

SCFS governs software configuration within SLGA™.

It does not replace requirements governance, software creation governance, dependency governance, software quality assessment, software security assessment, release authorization, deployment authorization, or broader lifecycle change management.

Within SLGA™:

Software Requirements Standard (SRS) governs software requirements and

their traceability.

Software Creation Standard (SCS) governs software creation methods,

artifacts, autonomous creation controls, composition, and development traceability.

Software Dependency Standard (SDS) governs dependency identification,

provenance, suitability, introduction, change, continuity, and risk.

Software Configuration Standard (SCFS) governs configuration baselines,

build configuration, artifact-version integrity, configuration change control, and reproducibility conditions.

Software Quality Standard (SQS) governs software quality requirements,

validation, testing, defect management, and quality acceptance.

Software Security Standard (SSS) governs software security requirements,

exposure, testing, vulnerabilities, and security response.

Software Release Standard (SRLS) governs release composition, readiness,

authorization, records, and withdrawal.

Software Deployment Standard (SDPS) governs deployment preparation,

authorization, production validation, monitoring, and rollback.

Software Change Management Standard (SCMS) governs broader software

lifecycle change.

Software Retirement Standard (SRTS) governs software retirement and

relevant transition conditions.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control, traceability, evidence, transitions, and assurance.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

operational conditions.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

configuration actions and operational boundaries.

AGI™ — Agent Governance Infrastructure: Relevant agent permissions and

tool-path governance.

SCFS preserves configuration-specific governance without replacing these

competent domains.

Canonical Boundary

SCFS begins when a material software configuration state, baseline, build configuration, artifact-version relationship, configuration change, or reproducibility condition requires governance recognition.

SCFS governs:

• Software configuration objects

• Configuration identity

• Configuration purpose

• Configuration scope

• Configuration items

• Configuration relationships

• Configuration baselines

• Baseline authority

• Baseline integrity

• Configuration versions

• Version relationships

• Build configuration

• Build-to-artifact correspondence

• Artifact-version integrity

• Configuration parameters

• Configuration values

• Defaults

• Overrides

• Environment-specific configuration

• Configuration assumptions

• Configuration composition

• Configuration consistency

• Configuration completeness

• Configuration constraints

• Declared configuration

• Actual configuration

• Declared–actual alignment

• Configuration changes

• Configuration change authority

• Configuration change execution

• Configuration change effects

• Configuration change traceability

• Configuration evidence

• Evidence competence

• Evidence integrity

• Configuration validation

• Configuration drift

• Configuration conflicts

• Configuration gaps

• Critical configuration gaps

• Configuration uncertainty

• Configuration reconstruction

• Configuration rollback references

• Build reproducibility conditions

• Configuration continuity

• Configuration governance states

SCFS does not independently determine:

• Software requirements acceptance

• Software creation authorization

• Dependency introduction authority

• Dependency suitability

• Complete software quality

• Software security certification

• Software release authorization

• Production deployment authorization

• Organization-wide operational continuity

• Legal compliance

• Legal liability

SCFS ends when the material software configuration conditions have been represented as a:

Governed Software Configuration State

with sufficient identity, baseline, version, build, artifact, parameter,

environment, authority, evidence, validation, uncertainty, and continuity information for competent downstream SLGA™ governance.

Canonical Closing Statement

Software does not become operational merely because its source code exists.

It must be assembled, built, versioned, configured, and established within an environment.

The same source code can produce different artifacts.

The same artifact can behave differently under different configuration parameters.

The same configuration file can produce different effective behavior when runtime overrides, environmental conditions, or external service settings change.

A software team may possess complete configuration documentation while the actual system operates under a different configuration.

An automated process may successfully apply a configuration change without establishing that the resulting state corresponds to the authorized change.

A build may succeed without sufficient evidence that it can be reproduced.

Declared Software Configuration ≠ Governed Software Configuration State.

Identical Software Artifact ≠ Identical Operational Configuration.

Configuration Parse Success ≠ Configuration Consistency.

Authorized Configuration Intent ≠ Verified Configuration Realization.

Repeated Build Success ≠ Demonstrated Build Reproducibility.

SCFS establishes the methodological requirement that software configuration must remain an identifiable and evidence-supported governance object rather than an assumed consequence of source code, build execution, or deployment automation.

This requirement applies whether configuration is established by human developers, automated infrastructure, AI coding assistants, or autonomous software systems.

The actor may change.

The configuration mechanism may change.

The software environment may change.

The requirement to establish what configuration actually exists—and under

which authority and conditions it exists—remains.

SCFS does not treat an approved baseline as proof of correct execution.

It does not treat successful compilation as proof of reproducibility.

It does not treat configuration documentation as proof of operational reality.

And it does not convert configuration validation into software release or deployment authorization.

Instead, it preserves the governed relationship between intended

configuration, established configuration, supporting evidence, material changes, and lifecycle continuity.

SCFS transforms software configuration from a collection of files,

parameters, versions, and build instructions into a governed software state — ensuring that software systems can establish not only how they were intended to be configured, but which configuration was actually established, how it was authorized, how its constituent elements relate, and whether that state remains sufficiently identifiable and verifiable throughout the software lifecycle.

Software configuration is not governed because it is documented, versioned, or successfully applied. It is governed when its identity, composition, authority, actual state, evidence, and continuity remain sufficiently established to support competent decisions about the software that depends upon it.

Module Architecture

→ CBM — Configuration Baseline Module

→ BMM — Build Management Module

→ AVIM — Artifact Version Integrity Module

→ CCCM — Configuration Change Control Module

→ BREM — Build Reproducibility & Environment Module

Related Documents

→ About Software Configuration Standard (SCFS)